



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3843 – SPECIAL TOPIC IN DATA ENGINEERING

SECTION 1

TUTORIAL AI ALGORITHM

NAME	MATRIC NUMBER
NURUL ASYIKIN BINTI KHAIRUL ANUAR	A23CS0162
LUBNA AL HAANI BINTI RADZUAN	A23CS0107
ANIS SAFIYYA BINTI JANAI	A23CS0049

LECTURER: DR. ARYATI BINTI BAKRI

Table of Contents

1.0 What is Image Classification?	3
2.0 What is Convolutional Neural Network (CNN)?	3
3.0 The Evaluation of Convolutional Neural Network (CNN)	3
4.0 Steps hand on in Python	4
5.0 What is the weakest in the current CNN model?	7
6.0 How to enhance the CNN model?	7
7.0 What is the evaluation based on CNN and new_CNN model?	8
8.0 Results and Discussion	8
ANN Results	9
CNN Results	10
Discussion	11
9.0 Reflection	12
9.1 NURUL ASYIKIN BINTI KHAIRUL ANUAR	12
9.2 LUBNA AL HAANI BINTI RADZUAN	12
9.2 ANIS SAFIYYA BINTI JANAI	12

1.0 What is Image Classification?

Image classification is the process of classifying an entire image. Images are anticipated to have just one class per image. Image classification models will take an image as an input and produce a prediction of the class the image belongs to as a result.

2.0 What is Convolutional Neural Network (CNN)?

A Convolutional Neural Network (CNN) is a type of neural network that is widely used for image classification and image recognition processes. From the tutorial, CNN is preferred for image processing since it can process image data more effectively and provide better classification results. CNNs are commonly used in applications that require object recognition, such as facial recognition systems and self-driving cars.

CNN is able to maintain the spatial layout of the image throughout the training process. In contrast to an Artificial Neural Network (ANN), input images need to be flattened into a one-dimensional vector at the beginning of the model. This is important because an image contains links between pixels such as edges, textures and object shape. By keeping this structure, CNN may extract relevant feature representations from image data.

The CNN model consists of several layers such as convolution layers, pooling layers, a flatten layer and dense layers. The convolution layers are used to extract features from image data while the pooling layers reduce the image dimensions and retain the important features of the image. Next, after the image has been processed, it is then transformed into a format in the flatten layer so that it can be used in the dense layer for classification.

3.0 The Evaluation of Convolutional Neural Network (CNN)

In this tutorial, the CIFAR-10 dataset was used for the Convolutional Neural Network (CNN) model, which has 60,000 images and 10 classes. The dataset was split into 50,000 training images and 10,000 testing images. Before training, all images were normalized by dividing them by 255.0 to improve the learning process.

The CNN architecture consisted of two convolutional layers, two max-pooling layers, one flatten layer, and two dense layers. The model was compiled using the Adam optimizer and sparse categorical cross-entropy loss function and trained for 10 epochs.

The evaluation of the CNN model was performed using the `cnn.evaluate()` function, which measures the model's performance on the test dataset by calculating the accuracy and loss of the test dataset. The CNN model achieved approximately 71% test accuracy with a loss value of around 0.89. This means that the model correctly classified about 71 out of every 100 images in the test dataset. The low value of loss shows that the predicted outputs were very close to the true class labels.

4.0 Steps hand on in Python

To complete this tutorial, Python code was used to process the dataset for image classification purposes. The first step for this tutorial involved the installation of all the necessary libraries, which are then imported into the environment before developing the image classification model, as shown in Figure 1. These libraries are required to support machine learning, data processing, and data visualization throughout the tutorial. For example, the TensorFlow library is needed for building and training the CNN models, while the Scikit-learn library is for model performance evaluation through confusion matrices.

```
!pip install tensorflow
!pip install scikit-learn
!pip install pandas

import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import sklearn
```

Figure 1: Install and Import Libraries

Next, the Cifar-10 dataset was loaded into the environment using `X_train`, `y_train`, `X_test`, and `y_test` attributes. After the dataset was loaded, the dataset dimensions of the four attributes were inspected using the `.shape` code, as in Figure 2. This is to get the information on the number of training images for each attribute, and also the image size.

```
(X_train, y_train), (X_test, y_test) = datasets.cifar10.load_data()

Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ————— 2s 0us/step

X_test.shape
(10000, 32, 32, 3)

X_train.shape
(50000, 32, 32, 3)

y_train.shape
(50000, 1)
```

Figure 2: Load Dataset and Check Dimension

Figure 3 demonstrates the reshaping process of the label data from a 2-dimensional array into a 1-dimensional array.

```
y_train[:5]

array([[6],
       [9],
       [9],
       [4],
       [1]], dtype=uint8)

y_train = y_train.reshape(-1,)
y_train[:5]

array([6, 9, 9, 4, 1], dtype=uint8)

y_test = y_test.reshape(-1,)
```

Figure 3: Reshape Label Data

In Figure 4, the class names were defined so that the numeric labels can be changed into readable object names. Then, a function was created in order to visualize the dataset along with the corresponding labels.

```
classes = [ 'airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck' ]

def plot_sample(X, y, index):
    plt.figure(figsize=(15,2))
    plt.imshow(X[index])
    plt.xlabel(classes[y[index]])
```

Figure 4: Class Definition and Visualizing Sample Images

Next, the data is normalized for a faster and better classification performance. After that, Figure 5 illustrates the building, compilation, and training processes of ANN model. When building ANN model, the image was converted into a 1D vector using `layers.Flatten`. As there were 10 classes, `layers.Dense(10)` indicates the output layer, which involves the 10 different classes, while the other `layers.Dense` codes indicate the hidden layer. The code “`activation='relu'`” is used to introduce non-linearity, while `softmax` is used to convert outputs into class probabilities. Then, the compilation of the ANN model includes optimizer updating weights and performance metric. `Ann.fit` code was used to train the ANN model for 5 epochs, which is the number of complete passes through the training dataset.

```
X_train = X_train / 255.0
X_test = X_test / 255.0
```

```
ann = models.Sequential([
    layers.Flatten(input_shape = (32, 32, 3)),
    layers.Dense(3000, activation = 'relu'),
    layers.Dense(1000, activation = 'relu'),
    layers.Dense(10, activation = 'softmax')
])

ann.compile(optimizer = 'SGD',
            loss = 'sparse_categorical_crossentropy',
            metrics = ['accuracy'])

ann.fit(X_train, y_train, epochs=5)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.p
super().__init__(**kwargs)
Epoch 1/5
1563/1563 ————— 79s 51ms/step - accuracy: 0.3537 - loss: 1.81
Epoch 2/5
1563/1563 ————— 82s 51ms/step - accuracy: 0.4262 - loss: 1.62
Epoch 3/5
1563/1563 ————— 75s 48ms/step - accuracy: 0.4562 - loss: 1.54
Epoch 4/5
1563/1563 ————— 80s 51ms/step - accuracy: 0.4793 - loss: 1.48
Epoch 5/5
1563/1563 ————— 77s 49ms/step - accuracy: 0.4952 - loss: 1.43
<keras.src.callbacks.history.History at 0x7f6b110676e0>
```

Figure 5: Data Normalization and ANN Model Process

From the previously trained model, the prediction of the classes for the test images was generated. The code `predict()` will generate predictions, while `argmax()` will select the highest probability class for the test images. Thus, the classification report was generated by calculating the precision, recall, and F1-score to evaluate the predictions. The evaluation of the trained CNN model will be using the unseen testing data, which then returns the test loss and accuracy.

```
from sklearn.metrics import confusion_matrix, classification_report
import numpy as np
y_pred = ann.predict(X_test)
y_pred_classes = [np.argmax(element) for element in y_pred]

print('classification report: \n', classification_report(y_test, y_pred_classes))
```

Figure 6: ANN Prediction

Figure 7 shows the evaluation of the model by creating a confusion matrix. This confusion matrix is used to differentiate the correct classification from the misclassifications.

```
import seaborn as sns

plt.figure(figsize = (14,7))
sns.heatmap(y_pred, annot = True)
plt.ylabel('Truth')
plt.xlabel('Prediction')
plt.title('Confusion matrix')
plt.show
```

Figure 7: Generate Confusion Matrix

5.0 What is the weakest in the current CNN model?

There are still several weaknesses that can be observed from the result, although the CNN model used in this tutorial performed better than the ANN model. The model achieved approximately 70% accuracy on the CIFAR10 dataset, which shows that a number of images were misclassified. This indicates that the model is reasonably effective but it is not yet highly reliable for making accurate predictions across all classes.

In addition, the current model architecture is relatively simple. The CNN used in the current model has only a few convolution and pooling layers with a small number of epochs which is 10 epochs. This limited training approach may not be enough for the model to properly understand the complicated features in the CIFAR10 dataset, which contains a variety of complex image categories. As a result, the model may underfit the data and fail to detect more detailed patterns.

Overall, the existing CNN model's weakest point is its limited performance due to a simple network architecture and training arrangement. Although it clearly illustrates CNN's effectiveness compared to ANN, more advancements would be required to produce predictions with greater accuracy and stability.

6.0 How to enhance the CNN model?

The CNN model can be improved by increasing the number of training epochs and layering the network. In the tutorial, the model was trained for only 10 epochs, which may not be sufficient to completely understand the CIFAR10 datasets' features. Training the model for more epochs, for example, 20 or 30 epochs could possibly enhance its learning and classification accuracy. In addition, another convolution layer with more filters might be added to help the model learn more information from the images.

Another technique to improve the model is to use data augmentation and dropout. Data augmentation techniques like picture flipping, rotation and zooming. It can generate multiple versions of the training images during training and allowing the model to perform better when tested with new images. Dropout layers can also be used to keep the model from memorizing the training data too much. These enhancements could help the CNN model achieve more accuracy and generate more accurate predictions.

7.0 What is the evaluation based on CNN and new_CNN model?

The performance of the original CNN model was compared with the improved new_CNN model using test accuracy and loss as evaluation metrics. The current CNN model achieved approximately 71% test accuracy with a loss value of 0.89. After enhancing the architecture by adding an additional convolutional layer, incorporating a dropout layer, and increasing the number of training epochs, the new CNN achieved a test accuracy of 75.94% and a loss value of 0.72.

The increase in accuracy from 71% to 75.94% means that the new_CNN was able to classify more images correctly. The loss is reduced from 0.89 to 0.72, indicating that the prediction of the model is closer to the true class labels in the meantime. These improvements indicate that the additional layers and training process led to the model learning more representative features from the CIFAR-10 dataset.

Overall, the new_CNN outperformed the original CNN model in both accuracy and loss. Therefore, the enhancements made to the CNN architecture were effective in improving image classification performance and producing more reliable predictions.

8.0 Results and Discussion

This section illustrates the results from the image classification using the 2 models, CNN and ANN. The classification performance based on the classification report and confusion matrix was also discussed in this section. Figure 8 shows the sample image that will be used for classification.

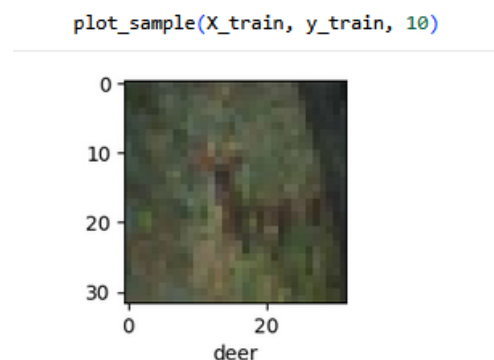


Figure 8: Display Sample Image

ANN Results

The ANN model was trained for 5 epochs using normalized images from the CIFAR-10 dataset. After training, the model achieved moderate classification performance. The classification report in Figure 9 showed varying precision, recall, and F1-scores across different object categories. As ANN used fully connected layers only, it struggled to capture spatial image features effectively.

```
... 313/313 ----- 5s 15ms/step
classification report:
      precision    recall  f1-score   support

 0         0.59      0.49      0.53       1000
 1         0.58      0.67      0.62       1000
 2         0.29      0.49      0.36       1000
 3         0.34      0.32      0.33       1000
 4         0.49      0.30      0.37       1000
 5         0.46      0.28      0.35       1000
 6         0.40      0.74      0.52       1000
 7         0.74      0.37      0.49       1000
 8         0.57      0.67      0.62       1000
 9         0.67      0.40      0.50       1000

 accuracy          0.47       10000
 macro avg         0.51      0.47      0.47       10000
 weighted avg      0.51      0.47      0.47       10000
```

Figure 9: ANN Classification Report

The ANN model achieved an overall classification accuracy of 47%. Among the ten CIFAR-10 categories, automobiles and ships recorded the highest F1-scores of 0.62, indicating better classification performance. Animal categories such as cats and dogs produced lower F1-scores of 0.33 and 0.35 due to visual similarities, making it harder for the ANN model to distinguish the two classes. The results demonstrate that the model has limited capability in extracting complex image features, which leads to moderate classification performance.

The confusion matrix illustrated the distribution of correct and incorrect classifications. Most predictions were concentrated along the diagonal as in Figure 10, indicating that correct classification and misclassifications mainly occurred between visually similar categories. These results highlight the limitations of ANN architectures in image classification tasks, as the flattening process removes important spatial information contained within images. Consequently, the model struggles to distinguish objects with similar visual characteristics.

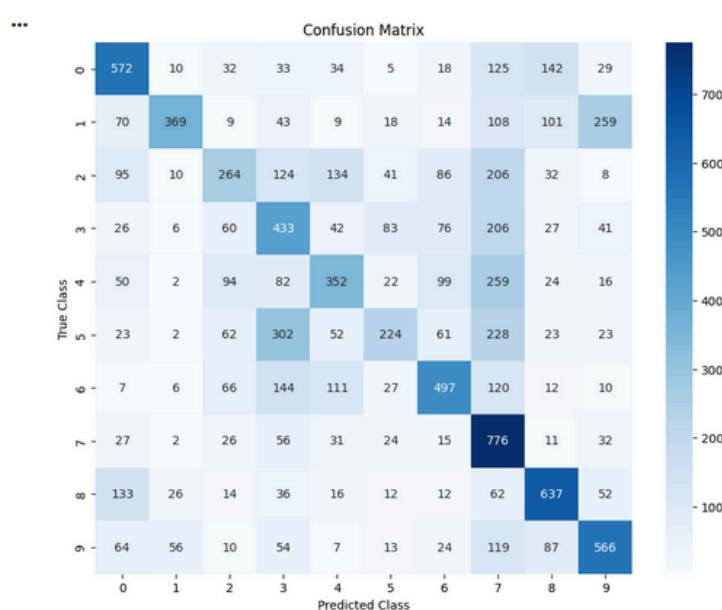


Figure 10: Confusion Matrix

CNN Results

The CNN model achieved significantly better performance than the ANN model. By utilizing convolutional and pooling layers, the CNN successfully extracted important image features such as edges, textures, and object shapes. The model demonstrated higher classification accuracy and more balanced performance across the ten CIFAR-10 classes as shown in Figure 11.

*** 313/313 6s 20ms/step

classification report:

	precision	recall	f1-score	support
0	0.75	0.72	0.74	1000
1	0.90	0.71	0.80	1000
2	0.64	0.58	0.61	1000
3	0.49	0.58	0.53	1000
4	0.66	0.70	0.68	1000
5	0.60	0.63	0.62	1000
6	0.82	0.73	0.78	1000
7	0.76	0.76	0.76	1000
8	0.75	0.84	0.79	1000
9	0.78	0.82	0.80	1000
accuracy			0.71	10000
macro avg	0.72	0.71	0.71	10000
weighted avg	0.72	0.71	0.71	10000

Figure 11: CNN Classification Report

Based on the CNN classification report in Figure 11, the model achieved an overall classification accuracy of 71%, representing a significant improvement over the ANN model. The average F1 scores are also 0.71, indicating a balanced classification. Among the ten CIFAR-10 categories, automobiles and trucks recorded the highest F1-scores of 0.80. This result suggests that CNN was highly effective in learning visual patterns for these categories. Overall, it is proven that CNN provides better classification performance on the CIFAR-10 dataset, and it is more effective than the ANN model for image recognition purposes.

Discussion

The tutorial used the CIFAR-10 dataset to verify the effectiveness of deep learning technology in image classification, as shown in Figure 12. Two different neural network architectures were developed and compared, including artificial neural networks (ANN) and convolutional neural networks (CNN).

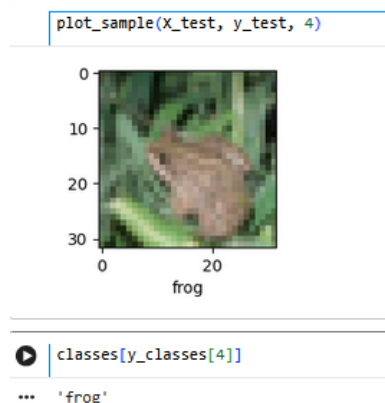


Figure 12: Display Sample Image and Classification

The ANN model flattens the image data into one-dimensional vectors before classification. Although this method can learn image patterns, it loses the spatial relationship between pixels, thus limiting the classification performance.

In contrast, the CNN model retains spatial information through convolutional operations. The convolutional layer automatically extracts important visual features, while the pooled layer reduces the computational complexity and prevents overfitting. Therefore, CNN has obtained higher classification accuracy.

The results show that the CNN architecture is more suitable for image recognition tasks than the traditional all-connected neural network. Future research directions can include increasing the number of convolutional layers, implementing data enhancement technology or increasing the number of training rounds to further improve classification performance.

9.0 Reflection

This section demonstrates the individual reflection for this tutorial.

9.1 NURUL ASYIKIN BINTI KHAIRUL ANUAR

This tutorial increased understanding of how Convolutional Neural Networks (CNN) are used for image classification compared to Artificial Neural Networks (ANN). Before the implementation, concepts like convolution, pooling and feature extraction were unclear, but the step-by-step process made it easier to understand how CNN handles CIFAR10 images. The difference in performance between CNN and ANN also demonstrates why CNN is more suitable for image classification because it keeps the image spatial structure and learns patterns better. Understanding how each layer was tough at first but after evaluating the model and its outcomes, it became clear. Finally, the future improvements to the model could include extra epochs, data augmentation and dropout to improve its performance on invisible images.

9.2 LUBNA AL HAANI BINTI RADZUAN

Through the tutorial, the basic concept of image classification using deep learning was learned. Practical experience of building a CNN model has been gained from this tutorial, in which the purpose of convolution, pooling, flatten, and dense layers in CNN architecture has been understood. After comparing the current CNN model with the new CNN model, the preservation of the spatial structure is known now to help improve classification accuracy. During the tutorial, challenges were found and one of them was the difficulty to select the most suitable hyperparameters, such as number of epochs and layers to ensure the model successfully improved, where the best value was found after a few experiments. In the future, an improvement to the model may be made by exploring the CNN model with an additional convolutional layer to enhance the model and increase the classification accuracy.

9.2 ANIS SAFIYYA BINTI JANAI

On completion of this tutorial, a basic understanding of image classification using CNN model is gained. This hands-on tutorial provides an in-depth understanding of training and testing the model in order to get the best outcome, which is a better accuracy image classification. Other than that, it was also shown in this tutorial that CNN was a better model than ANN in image classification, as it provides a better overall performance. This was due to how CNN handles the image while maintaining its spatial structure through convolutional operations, which leads to a more balanced and accurate classification. One of the challenges faced in completing this tutorial is understanding each layer and its purpose, but after completing the tutorial, it is made clear the purpose of each layer, including the convolutional layers and pooling layers. For future improvement, the model could implement a data augmentation method and train for additional epochs to ensure accuracy and improve classification performance.